Inference-Centric AI Systems

Data Movement, KV Caches, Scheduling, Speculation, Quantization, Phase Disaggregation, MoE Serving, RL Rollouts, and Specialized Hardware

Independent Technical Paper

August 2026

Central thesis

The decisive resource in large-model inference is not arithmetic alone, but the movement, placement, reuse, and synchronization of model state. The most useful unifying metric is therefore bytes moved per useful token under latency and quality constraints.

Scope

This paper is a systems synthesis and analytical framework. Equations are intended as engineering cost models; benchmark outcomes depend on model architecture, workload distributions, hardware, kernels, and service-level objectives.

Abstract

Large-model inference is becoming the dominant systems problem in modern artificial intelligence. Autoregressive generation converts a single model invocation into a long-lived, stateful process whose performance depends on repeated weight access, a growing key-value (KV) cache, irregular request arrivals, variable prompt and output lengths, and strict first-token and inter-token latency objectives. The resulting workload is neither a conventional batch computation nor a simple online transaction service. It is a dynamic data-orchestration problem spanning memory hierarchy, kernels, scheduling, networking, and hardware specialization. This paper develops a unified account of the main inference techniques: KV caching and paged attention, continuous batching, chunked prefill, speculative decoding, weight/activation/KV quantization, prefill-decode disaggregation, latency-throughput optimization, and sparse mixture-of-experts (MoE) serving. It then connects these mechanisms to reinforcement-learning post-training, where rollout generation is itself a large inference workload and frequently determines end-to-end training throughput. The analysis argues that bytes moved per useful token, rather than peak operations per second, is the most revealing first-order metric. Historical circuit data illustrate why remote data movement can consume orders of magnitude more energy than local arithmetic, while current inference systems show that the exact bottleneck can shift between memory, compute, and network as batching and kernel overlap improve. The practical goal is therefore not to label all inference “memory-bound,” but to construct a phase- and workload-specific resource model and deliberately move the bottleneck to the least costly resource. The paper concludes with a reference architecture and a hardware-software co-design agenda: larger and software-managed on-chip memories, low-precision matrix and attention engines, first-class KV-cache operations, efficient collectives for MoE, high-bandwidth scale-up fabrics, near-memory data transforms, and separate optimization envelopes for prefill, decode, and RL rollout generation. The strategic implication is that future inference platforms will be judged by useful tokens per joule and SLO-compliant goodput, not by nominal peak compute alone.

Keywords— large-model inference, KV cache, PagedAttention, continuous batching, speculative decoding, quantization, prefill-decode disaggregation, TTFT, goodput, mixture of experts, RL rollouts, inference accelerators, data movement.

Contents

1. Introduction and central argument
2. Workload model and performance vocabulary
3. Data movement as the organizing constraint
4. KV cache, attention state, and paged memory management
5. Continuous batching and phase-aware scheduling
6. Speculative decoding
7. Quantization as data-movement compression
8. Prefill/decode disaggregation
9. TTFT versus throughput: the service frontier
10. Serving sparse mixture-of-experts models
11. Why inference and reinforcement learning are converging
12. Specialized inference hardware
13. A reference inference architecture
14. Measurement and optimization methodology
15. Research agenda
16. Conclusion

References

1. Introduction and Central Argument

Transformer inference begins with a deceptively simple operation: given a sequence of tokens, compute the probability distribution of the next token [1]. In an interactive service, however, that operation is embedded in a stateful loop. A request first processes its prompt, allocates attention state, emits a first token, and then repeatedly re-enters the model while competing with other requests for memory capacity, memory bandwidth, compute units, and network links. Prompt lengths and output lengths are not known in advance, requests arrive asynchronously, and the service may be required to satisfy both a first-response deadline and a steady token cadence.

The central systems mistake is to reduce this workload to a single scalar such as peak FLOP/s, tokens/s, or model size. Each is informative but incomplete. Peak FLOP/s ignores whether operands can be delivered to arithmetic units. Aggregate tokens/s can conceal unacceptable tail latency. Model size ignores the transient KV state and communication induced by tensor, pipeline, expert, or phase parallelism. A useful inference design must instead reason about the complete path of every byte and every token.

This paper adopts a data-centric thesis: the main optimization object is the movement and residency of model weights, activations, KV blocks, routing metadata, sampled tokens, and distributed-training state. Arithmetic remains essential, but its efficiency is often determined by whether the surrounding system reuses data locally, overlaps independent resource streams, compresses transfers, and schedules work at the right granularity. The same principle explains why paged KV management, continuous batching, quantization, disaggregation, MoE placement, and specialized hardware are not separate topics. They are alternative ways of changing where bytes live, how often they move, and how much useful work is obtained from each movement.

A second argument is that the inference bottleneck is movable. At low decode batch sizes, repeated model-weight reads frequently dominate. At long context, KV traffic and capacity can dominate. Large prefill batches may become compute-bound. MoE models can become network-bound. Aggressive quantization can relieve memory bandwidth only to expose dequantization, reduction, or scheduling overhead. Recent throughput-oriented systems also show that end-to-end inference can be compute-bound when memory, compute, and network operations are carefully overlapped [21]. Therefore, “inference is memory-bound” is best treated as a starting hypothesis, not a universal conclusion.

A third argument is that reinforcement learning makes inference a training primitive. In RLHF and reinforcement learning with verifiable rewards, the learner repeatedly samples multiple long responses from a changing policy, interacts with tools or environments, scores trajectories, and updates model weights. The rollout stage is inference at large scale, but with additional requirements: policy-version consistency, controlled sampling diversity, frequent weight refresh, trajectory metadata, and tight coupling to optimizer throughput. The boundary between serving and training systems is consequently disappearing [27], [28].

Design objective: Maximize useful tokens or useful trajectories per joule and per dollar while meeting first-token, inter-token, end-to-end latency, quality, and fairness constraints.

1.1 Contributions of this paper

- A unified analytical model connecting TTFT, time per output token, throughput, goodput, memory footprint, arithmetic intensity, and energy.
- A phase-specific explanation of KV paging, continuous batching, speculation, quantization, and prefill/decode disaggregation.
- A serving model for sparse MoE networks that treats expert placement, token routing, all-to-all communication, and load imbalance as first-class inference costs.
- A systems interpretation of RL rollouts as an inference fleet with policy synchronization and trajectory-production objectives.
- A hardware-software co-design agenda centered on local state, low precision, dataflow, collectives, and specialized engines.

2. Workload Model and Performance Vocabulary

2.1 Prefill and decode are different computational phases

A decoder-only Transformer request has two operational phases. During prefill, the model processes all prompt tokens and builds the initial KV cache. The computation exposes parallelism across tokens, layers, heads, and batch elements. Depending on prompt length and batch size, dense matrix multiplications can be large enough to use accelerator compute efficiently. During decode, each active sequence contributes one new query token per iteration. The model reuses its existing KV cache, appends one key and one value per layer, and produces the next-token distribution. Decode has much less sequence-level parallelism and repeatedly streams weights and KV state.

This asymmetry is architectural. Prefill benefits from compute-oriented parallelism, while decode benefits from weight reuse across a large continuous batch, efficient KV reads, low launch overhead, and stable token cadence. Treating both phases identically couples incompatible optimization goals and can cause a long prompt to stall ongoing decodes.

Dimension	Prefill	Decode
Input processed per request	Many prompt tokens	One newly generated token per iteration
Parallelism	High token and matrix parallelism	Primarily batch, head, layer, and model parallelism
Typical pressure	Compute, activation traffic, prompt attention	Weight bandwidth, KV bandwidth/capacity, launch and synchronization
Primary user metric	Time to first token (TTFT)	Time per output token (TPOT) / inter-token latency
Batching risk	Long prompts can monopolize an iteration	Small batches underuse compute and weight reuse
Natural hardware bias	High compute density	High bandwidth, capacity, low overhead

2.2 Core metrics

A production service should report a vector of metrics rather than a single throughput number. TTFT measures the interval from request arrival to the first returned token. TPOT or inter-token latency measures the cadence after the first token. End-to-end latency depends on both. Throughput measures generated tokens or completed requests per second, whereas goodput counts only the work that meets specified service-level objectives. Tail percentiles are indispensable because batching and queueing policies often improve the mean while harming a small but important set of requests.

$$\mathrm{TTFT} = T_{\mathrm{queue,prefill}} + T_{\mathrm{prefill}} + T_{\mathrm{sample,first}} \quad (1)$$

$$T_{\mathrm{E2E}} \approx \mathrm{TTFT} + (N_{\mathrm{out}} - 1) \cdot \mathrm{TPOT}_{\mathrm{avg}} \quad (2)$$

$$\mathrm{Goodput} = \text{completed requests satisfying all SLOs / second} \quad (3)$$

The approximation in (2) is intentionally simple. Real services exhibit variable TPOT, network streaming delay, stop-condition checks, tool calls, and preemption. Nevertheless, it highlights a critical point: an optimization that lowers average decode cost but increases queueing can improve total tokens/s while worsening TTFT. Conversely, reserving excessive capacity for immediate prefill can produce excellent TTFT at poor device utilization.

2.3 Arithmetic intensity and the moving bottleneck

The roofline model relates attainable performance to arithmetic intensity, defined as operations performed per byte transferred from a constraining memory level [2]. For a kernel with peak compute C_{peak}, effective bandwidth BW, and arithmetic intensity I, the attainable rate is bounded by the smaller of compute capacity and bandwidth-scaled intensity.

$$\mathrm{Performance} \le \min(C_{\mathrm{peak}}, \mathrm{BW} \cdot I) \quad (4)$$

Decode batching raises effective intensity because a model weight loaded once can be reused across more sequences. Quantizing weights lowers bytes per weight and can raise intensity again. Fusing kernels avoids intermediate writes. KV compression lowers attention traffic. Operation overlap can make the aggregate pipeline compute-bound even when

individual attention or communication operators are bandwidth-bound. The correct question is therefore: which resource constrains the current workload after all reuse and overlap are accounted for?

3. Data Movement as the Organizing Constraint

The energy and latency cost of a datum depends strongly on where it resides. Registers and small local memories are close to arithmetic units; large on-chip SRAMs are farther away; high-bandwidth memory is off-core; host DRAM, storage, and remote accelerators add progressively more serialization, protocol, and transport overhead. Historical circuit estimates show that a DRAM read can cost hundreds to thousands of times more energy than a simple arithmetic operation, depending on the arithmetic chosen and the technology assumptions [3], [4]. The often-repeated “approximately one thousand times” statement should be understood as an order-of-magnitude design warning, not a timeless physical constant.

The enduring principle is more important than any single ratio: computing on a value after it has been placed near an arithmetic unit is cheap relative to repeatedly transporting that value through a deep hierarchy. Inference amplifies this problem because model weights and KV state are revisited for every generated token. A small inefficiency multiplied by billions of tokens becomes a fleet-level power and capital cost.

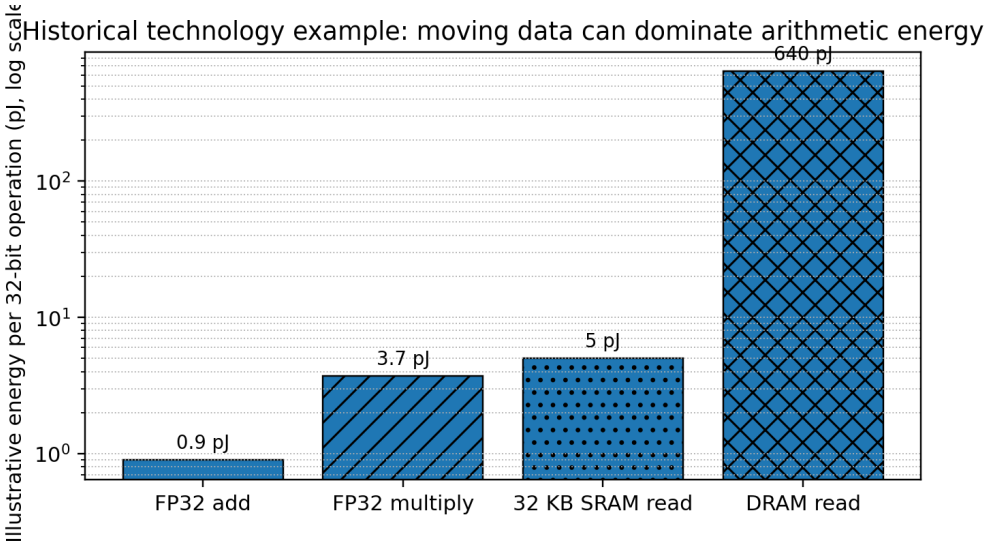


Figure 1. Illustrative historical energy hierarchy. Exact values change with process, voltage, memory technology, access granularity, and packaging, but locality remains a dominant design principle.

3.1 What moves during inference

- Weights: dense or expert parameters streamed from HBM or sharded across devices. Weight traffic is amortized by batch reuse.
- Activations: intermediate tensors between operators and layers. Fusion and tiling reduce write-back and reload.
- KV cache: per-sequence, per-layer attention state that grows with context length and is reread during decode.
- Collective data: tensor-parallel reductions, pipeline transfers, expert dispatch/combine traffic, and synchronization metadata.
- Control data: block tables, routing indices, sequence lengths, sampling state, stop conditions, and scheduler decisions.
- RL data: sampled tokens, log probabilities, masks, rewards, tool transcripts, trajectories, and frequently refreshed policy weights.

3.2 Bytes per useful token

A practical first-order metric is bytes moved per useful output token (BPUT). “Useful” excludes rejected speculative tokens, padding, duplicated recomputation, late outputs that violate the SLO, and trajectories discarded because they use an obsolete policy version. The metric can be measured at several boundaries: HBM, interconnect, host-device, rack network, or storage.

$$\text{BPUT} = (\text{weight bytes} + \text{KV bytes} + \text{activation bytes} + \text{network bytes} + \text{control bytes}) / \text{useful output tokens} \quad (5)$$

BPUT is not a replacement for latency or compute utilization. It is a diagnostic that makes hidden waste visible. A design may reduce HBM bytes but increase network bytes; save weight traffic but expand KV state; improve raw tokens/s but generate more rejected speculative work. The system should minimize a weighted movement cost whose coefficients reflect energy, latency, congestion, and monetary cost at each hierarchy level.

$$\text{Movement cost} = \Sigma_h \lambda_h \cdot \text{Bytes}_h \quad (6)$$

Here h indexes hierarchy boundaries and λ_h captures the marginal cost of moving a byte across that boundary. Specialized inference hardware is fundamentally an attempt to reduce λ_h, reduce Bytes_h, or increase the useful work performed before the byte moves again.

4. KV Cache, Attention State, and Paged Memory Management

4.1 Why KV caching is necessary—and expensive

Without a KV cache, generating token t would require recomputing keys and values for all preceding tokens. Caching converts this repeated computation into persistent state. For L Transformer layers, sequence length S, n_{kv} key/value heads, head dimension d_h, element size b bytes, and batch sequences indexed by i, the approximate KV footprint is:

$$M_{KV} = 2 \cdot L \cdot n_{kv} \cdot d_h \cdot b \cdot \Sigma_i S_i \quad (7)$$

The factor of two accounts for keys and values. Standard multi-head attention uses one KV head per query head; multi-query attention and grouped-query attention reduce n_{kv} and therefore directly reduce KV memory and decode bandwidth [8]. Equation (7) omits allocator metadata, alignment, temporary workspaces, replication under parallelism, and any retained prefixes. It nevertheless shows the linear scaling with context length, concurrency, number of layers, and KV-head count.

KV state is unusually difficult to manage because its lifetime and final size are unknown at admission. Preallocating for the maximum output wastes memory. Growing contiguous buffers requires over-reservation or costly relocation. Variable-length sequences fragment the heap, and parallel sampling may duplicate large prefixes. These effects reduce the number of requests that fit concurrently, which in turn reduces weight reuse and throughput.

4.2 Paged attention

PagedAttention applies the virtual-memory idea to attention state: a sequence sees a logically contiguous list of KV blocks, while physical blocks can reside non-contiguously in accelerator memory [7]. A block table maps logical positions to physical blocks. The allocator adds blocks on demand as tokens are generated, and unused blocks return to a global pool. Internal fragmentation is limited to the unfilled portion of the final block rather than an entire over-reserved maximum sequence.

Paging also enables reference-counted sharing. Requests with a common prompt prefix can point to the same physical blocks. Parallel samples can share the prompt and diverge through copy-on-write. Eviction, migration, and tiering become block operations instead of whole-sequence operations. The block size is a systems parameter: small blocks reduce internal waste and migration granularity but increase metadata, lookup, and kernel overhead; large blocks do the opposite.

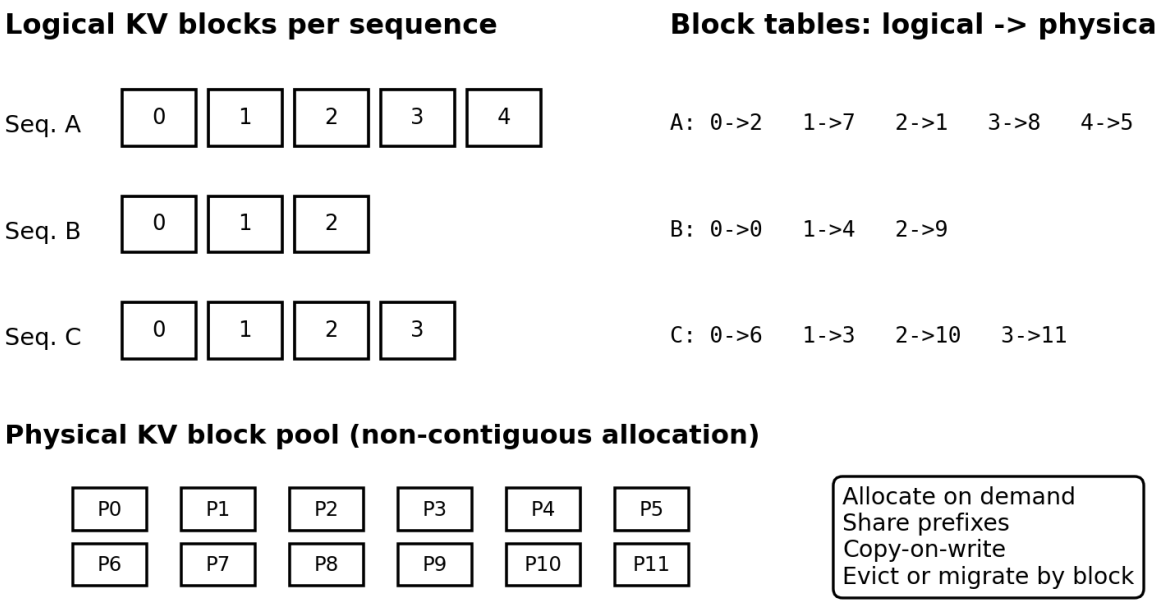


Figure 2. Paged KV management separates logical sequence order from physical allocation, enabling demand allocation, sharing, copy-on-write, migration, and block-level eviction.

4.3 Prefix reuse and semantic locality

Many production workloads repeatedly use the same system prompt, retrieved document prefix, few-shot examples, tool schema, or conversation history. Prefix caching converts repeated prefill computation into a lookup and KV reuse operation. A radix tree or related prefix index can organize cached token sequences, while the KV directory tracks physical block locations and reference counts. SGLang's runtime demonstrates how prefix-aware scheduling and RadixAttention can exploit this structure in complex language-model programs [10].

Prefix caching changes routing. The nearest idle accelerator may not be the cheapest destination if another accelerator already owns most of the required prefix. A cache-aware scheduler should compare queueing delay against avoided prefill compute and transfer cost. At cluster scale, KV placement becomes analogous to distributed storage placement, with hot prefixes, replication, consistency, and eviction policies.

4.4 KV quantization, offload, and selective retention

KV compression attacks both capacity and bandwidth. KIVI, for example, uses asymmetric low-bit treatment for keys and values based on their different distributions [9]. More generally, a service can use lower precision for older tokens, retain a recent full-precision window, or combine quantization with sparse retention. Offloading to CPU DRAM or SSD expands capacity but adds transfer latency and may be beneficial only when recomputation is more expensive than retrieval.

The key design rule is to preserve attention quality while making the compressed representation directly consumable by the attention kernel. A representation that saves storage but requires expensive unpacking, irregular gathers, or full-precision reconstruction can lose its theoretical advantage. KV format, page layout, quantization group, and kernel tile should therefore be co-designed.

5. Continuous Batching and Phase-Aware Scheduling

5.1 From request-level batching to iteration-level scheduling

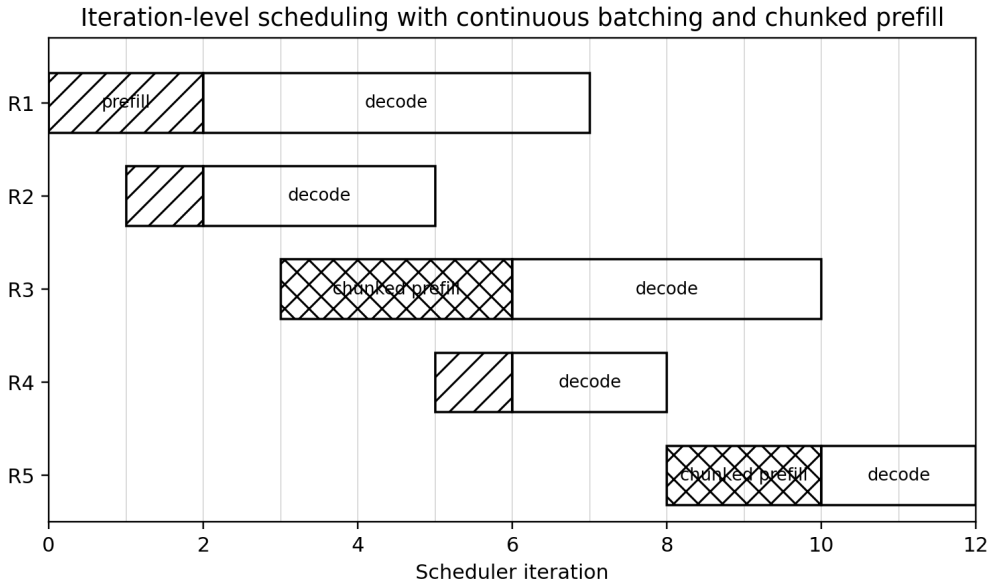
Static batching groups requests and runs the batch until every request finishes. For autoregressive generation, this creates severe head-of-line blocking: short requests wait for the longest request, completed slots remain idle, and new requests cannot join. Iteration-level scheduling instead rebuilds the active batch at each decode step. Completed requests leave immediately, new requests enter, and the scheduler can preempt or reprioritize sequences. Orca formalized this shift and selective batching for generative Transformer serving [6].

Continuous batching is not merely a queue implementation. It is the mechanism that turns unpredictable request lengths into stable accelerator occupancy. Its effectiveness depends on KV memory availability, maximum token budget per iteration, admission policy, preemption cost, and fairness. A scheduler that maximizes the current batch can over-admit long sequences and trigger future KV exhaustion; a scheduler that reserves too much space can leave bandwidth idle.

5.2 Chunked prefill and stall-free schedules

Mixing prefill and decode creates a conflict. A large prefill can efficiently use compute but delay every active decode, producing an inter-token latency spike. Chunked prefill divides a long prompt into bounded work units and combines at most a controlled amount of prefill with decode tokens in each iteration. Sarathi-Serve uses chunked prefill and stall-free scheduling to navigate the throughput-latency tradeoff [11].

The chunk size controls a three-way compromise. Larger chunks improve matrix efficiency and finish prompts sooner; smaller chunks bound decode interference and improve admission responsiveness; excessively small chunks increase launch, bookkeeping, and repeated kernel overhead. The correct size depends on model parallelism, prompt distribution, target TPOT, and whether the system is colocated or phase-disaggregated.



Completed requests leave immediately; newly admitted work joins the next iteration. Chunking bounds interference from long prompts.

Figure 3. Continuous batching admits and retires requests at iteration boundaries. Chunked prefill limits the duration for which a long prompt can delay active decodes.

5.3 Scheduling objectives

A production scheduler is multi-objective. It must protect TTFT, TPOT, and end-to-end deadlines; maximize goodput; avoid KV-cache deadlock; provide fairness among tenants; and account for prefix locality, speculative decoding state, LoRA adapters, or MoE expert placement. Priority should therefore be derived from slack and resource cost rather than arrival time alone.

$$\text{Slack}_i = \text{Deadline}_i - \text{CurrentTime} - \text{PredictedRemainingService}_i \quad (8)$$

A negative or small slack suggests urgency, but predicted remaining service is uncertain because output length is unknown. Robust schedulers combine online length prediction, conservative memory reservation, age-based fairness, and admission backpressure. For RL rollouts, the scheduler may additionally group requests by policy version, sampling configuration, or environment affinity.

6. Speculative Decoding

6.1 Draft, verify, accept

Autoregressive decoding is sequential because token t+1 depends on token t. Speculative decoding reduces the number of expensive target-model iterations by using a cheaper draft mechanism to propose several future tokens. The target model

verifies the proposed block in parallel. With the appropriate acceptance and correction rule, the resulting samples preserve the target distribution [12], [13].

Let γ be the number of draft tokens proposed and α an approximate independent acceptance probability per position. A useful approximation for the expected number of output tokens produced by one target verification pass is:

$$E[\text{tokens per verification}] \approx \Sigma_{i=0}^{\gamma} \alpha^i = (1 - \alpha^{(\gamma+1)}) / (1 - \alpha) \quad (9)$$

The actual speedup depends on draft cost, target verification efficiency, acceptance correlations, sampling settings, vocabulary projection, communication, and batching. If c_T is the target verification cost and c_D is the cost per draft step, a simplified speedup estimate is:

$$\text{Speedup} \approx E[\text{tokens per verification}] \cdot c_{T,1} / (c_{T,\gamma} + \gamma \cdot c_D + c_{control}) \quad (10)$$

Here $c_{T,1}$ is the cost of one ordinary target decode step and $c_{T,\gamma}$ is the cost of verifying the block. Verification is attractive because several candidate positions can be processed with larger matrix operations and a single weight stream, but it is not free. Low acceptance wastes draft work and target computation; long draft blocks increase the probability of early rejection.

6.2 System interactions

- Continuous batching: different requests accept different numbers of tokens, creating variable advancement and more complex batch compaction.
- KV management: accepted tokens append to the target KV cache; rejected suffix state must be discarded or never committed.
- Quantization: a quantized draft model may be faster but less aligned with the target, reducing acceptance. The best draft is not necessarily the smallest.
- MoE: target verification can activate different experts for each proposed position, increasing token-routing burstiness.
- Disaggregation: draft and target services may be colocated for low latency or separated to share a draft fleet, but remote draft proposals add transport and queueing.
- RL rollouts: distribution-preserving speculation can accelerate sampling without intentionally changing the policy distribution, but implementation numerics, policy refresh, and rejection bookkeeping must be audited.

6.3 Beyond a separate draft model

Drafts can come from a smaller model, early-exit layers, multiple prediction heads, retrieved continuations, prompt-specific caches, or learned tree proposals. The unifying requirement is inexpensive proposal generation with high acceptance under the target. A serving system should tune draft length and strategy online using measured acceptance, queue state, and target saturation. Under high target load, extra draft work may be worthwhile; under low load or strict single-request latency, ordinary decoding may be simpler and faster.

Operational rule: Enable speculation only when measured accepted tokens per target pass exceed the added draft, verification, memory, and scheduling cost. Acceptance rate alone is not a sufficient metric.

7. Quantization as Data-Movement Compression

7.1 What can be quantized

Quantization maps values to lower-precision representations. In inference, it can reduce model capacity requirements, increase effective memory bandwidth, increase cache residency, lower communication volume, and use higher-throughput low-precision arithmetic. The major targets are weights, activations, KV cache, and distributed communication. Each has different statistics and different sensitivity.

Weight-only quantization such as GPTQ and AWQ is attractive for decode because it reduces the bytes streamed for model parameters while leaving activations at higher precision [15], [16]. SmoothQuant transforms activation outliers into a form that enables efficient W8A8 execution [14]. KV quantization addresses long-context capacity and attention bandwidth [9]. System co-designs such as QServe combine low-bit weights, activations, and KV state while optimizing kernel data paths and dequantization overhead [17].

Mode	Primary benefit	Typical risk	When it helps most

W8A8 / FP8	Lower weight and activation traffic; fast matrix engines	Outliers, calibration, accumulation error	Compute-heavy prefill and large batches
W4A16	Large weight compression with moderate activation change	Dequantization overhead; limited tensor-core mapping	Low-to-medium decode batches where weights dominate
W4A8	Weight and activation compression	Kernel complexity and partial-sum handling	Cloud serving with hardware-friendly fused kernels
KV4 / KV2	More concurrent sequences; less attention traffic	Long-context quality loss; unpacking overhead	Long contexts and KV-limited decode
Quantized collectives	Lower network bytes	Error accumulation and conversion cost	Tensor/expert parallel deployments

7.2 Quantization changes the bottleneck

The benefit of quantization is not proportional to the reduction in bit width. A fourfold storage reduction does not guarantee a fourfold speedup. Packing, scale loads, zero-point handling, dequantization, conversion, alignment, and unsupported instruction paths can consume the saved time. At high batch sizes, a formerly bandwidth-bound operator may become compute-bound. At very low precision, reductions and softmax may remain at higher precision and dominate.

Quantization should therefore be evaluated end to end and by phase. Weight-only INT4 may transform decode economics but provide a smaller prefill gain if dequantization prevents efficient large GEMMs. KV4 may increase batch capacity and indirectly improve weight reuse, producing a second-order throughput gain larger than the attention-kernel speedup alone. Conversely, a format that reduces bytes but fragments the serving runtime into many unfused kernels can worsen TTFT.

7.3 Quality and numerical contracts

Inference quantization is constrained by a quality contract, not merely average benchmark accuracy. The contract can include perplexity, task accuracy, calibration, rare-token behavior, safety filters, tool-call formatting, long-context retrieval, and sampling stability. RL rollouts add another dimension: small distribution changes can alter exploration, reward variance, and policy updates. A deployment should validate both endpoint quality and the statistical properties of generated trajectories.

Mixed precision is usually more valuable than uniform minimum precision. Sensitive layers, outlier channels, embeddings, logits, normalization, routing, or recent KV windows may remain at higher precision. The hardware implication is that efficient inference engines need fast conversion, flexible accumulation, per-group scales, and fused mixed-precision data paths rather than one fixed numeric format.

8. Prefill/Decode Disaggregation

8.1 Motivation

Colocating prefill and decode on the same devices maximizes local state reuse but couples two phases with different resource profiles. Long prefills interfere with decode cadence; the same parallelism configuration must serve both phases; and independent scaling is impossible. DistServe and Splitwise show that separating the phases can improve goodput, cost, or power by assigning each phase to a better-suited pool [18], [19].

Disaggregation introduces a new cost: the KV cache produced by prefill must move to the decode worker. The transfer includes all layers and prompt positions, so it can be large. The decision is therefore a queueing-and-transport tradeoff, not a categorical rule.

$$T_{KV\text{-}transfer} \approx M_{KV,prompt} / BW_{effective} + T_{setup} + T_{queue,network} \quad (11)$$

Disaggregate when saved interference + queueing + specialization gain > KV transfer penalty + duplication cost (12)

8.2 Benefits

- Independent provisioning: scale prefill capacity with prompt-token arrival rate and decode capacity with generated-token occupancy.
- Phase-specific parallelism: use tensor/pipeline configurations and batch shapes suited to each phase.
- Hardware specialization: assign compute-dense accelerators to prefill and bandwidth/capacity-efficient devices to decode.
- SLO isolation: protect decode cadence from long prefills and control TTFT separately.
- Fault and upgrade domains: update or autoscale one pool without draining all active sessions.

8.3 Costs and failure modes

- KV transfer can dominate short outputs: the service pays a large one-time movement cost but amortizes it over few decode tokens.
- Model replication increases total weight capacity and can lower utilization if phase ratios vary rapidly.
- Network contention and topology matter; advertised link bandwidth is not the same as end-to-end KV transfer bandwidth.
- Placement becomes stateful: the scheduler must match prefill outputs with decode capacity and avoid hot receivers.
- Retries require ownership rules for partially transferred KV blocks and idempotent metadata updates.
- Heterogeneous precision or layout between pools may require format conversion on the critical path.

8.4 KV-centric disaggregated memory

A broader design treats KV state as a cluster-level object rather than private device memory. Mooncake combines separated prefill and decode clusters with a distributed KV tier spanning otherwise underused CPU memory and storage, coordinated by a KV-aware scheduler [20]. This architecture is especially relevant for long-context chat, prefix reuse, and bursty traffic. It also reframes the system: GPUs become compute clients of a distributed state service.

The extension is powerful but demands storage-system discipline: naming, block checksums, versioning, admission based on transfer bandwidth, hot-prefix replication, eviction, and topology-aware placement. The fastest cache hit is still a local hit; the global tier is valuable when it avoids expensive recomputation or increases accelerator occupancy enough to cover transport.

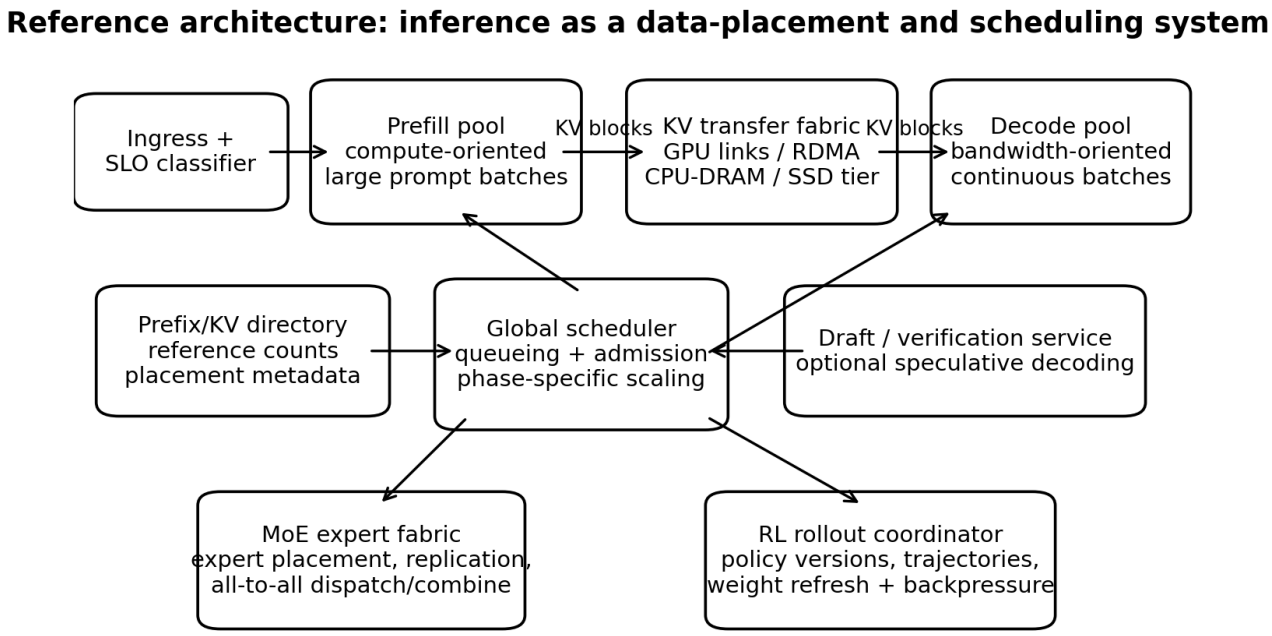


Figure 4. A phase-disaggregated, KV-centric inference architecture. Prefill and decode pools can scale independently, while a global scheduler reasons about SLOs, state location, transfer cost, speculation, MoE placement, and RL policy versions.

8.5 Partial and topology-aware disaggregation

Full separation is not always optimal. A service may colocate prefill and decode within a fast scale-up domain, disaggregate only overflow traffic, perform temporal phase separation on commodity clusters, or keep short prompts local while exporting long prompts. The decision can be made per request using predicted output length, prompt KV size, current queue state, and network locality. The best architecture is often hierarchical: local phase splitting within a node, rack-level KV exchange for load balance, and storage-tier reuse for long-lived prefixes.

9. TTFT Versus Throughput: The Service Frontier

9.1 Why the objectives conflict

Throughput improves when more requests share each weight load and when larger matrix operations use the accelerator efficiently. TTFT improves when a new request starts immediately. These objectives conflict because waiting creates a larger batch but adds queueing delay. Decode TPOT creates another constraint: a long prefill or oversized verification batch can improve aggregate efficiency while delaying the next token of every active sequence.

The correct output of an inference benchmark is therefore a Pareto frontier rather than a single maximum-throughput point. Each point corresponds to an admission limit, batching window, token budget, prefill chunk size, parallelism plan, and possibly a different hardware pool. A service chooses an operating point based on workload class and SLO.

9.2 Queueing-aware decomposition

$$\text{TTFT} = \text{admission delay} + \text{queueing delay} + \text{service time} + \text{transfer time} \quad (13)$$

$$\text{TPOT_p99} = \text{kernel time} + \text{scheduler gap} + \text{interference} + \text{synchronization} + \text{network streaming} \quad (14)$$

Optimizing only service time can be misleading. A kernel speedup may allow the scheduler to admit more work, which increases queueing and leaves tail TTFT unchanged. Conversely, a strict concurrency cap can improve p99 while wasting capacity during benign workload periods. Goodput optimization incorporates the SLO directly: the best configuration is the one that completes the most compliant requests, not the one that starts the most work.

9.3 Practical controls

Control	Improves	Can harm	Use
Larger decode batch	Weight reuse, throughput	Queueing, KV pressure, fairness	Increase until SLO or memory becomes binding
Batching window	Matrix efficiency	TTFT	Use microsecond/millisecond adaptive windows by load
Prefill chunk size	Prompt completion efficiency	Decode cadence if too large; launch overhead if too small	Tune against p99 TPOT
Concurrency cap	Tail latency, memory safety	Peak throughput	Set from KV and service-time distributions
Priority / deadline scheduling	SLO compliance	Starvation risk	Combine slack with aging
Speculation depth	Tokens per target pass	Rejected work and memory	Adapt to acceptance and target load
Phase disaggregation ratio	Isolation and specialization	Transfer and replication cost	Autoscale from prompt/output token rates

9.4 Cost-normalized metrics

For fleet planning, tokens/s/device is insufficient because devices differ in acquisition cost, power, memory capacity, and networking. More informative metrics include SLO-compliant tokens per joule, requests per dollar-hour, and useful trajectories per accelerator-hour. Capacity planning should use real prompt/output distributions and arrival burstiness. Synthetic fixed-length benchmarks are useful for kernel characterization but can hide allocator fragmentation, queueing, prefix locality, and tail effects.

$$\text{Efficiency_SLO} = \text{useful tokens meeting SLO} / (\text{energy or cost}) \quad (15)$$

10. Serving Sparse Mixture-of-Experts Models

10.1 Sparse compute does not mean sparse systems cost

A sparse MoE layer contains multiple feed-forward experts and a router that selects a small top-k subset for each token. Only the selected experts perform arithmetic, allowing the total parameter count to grow faster than active compute. Mixtral is a representative sparse decoder architecture in which each token activates a subset of available experts [25].

The serving challenge is that inactive parameters still require storage, selected experts may reside on different devices, and token routing induces all-to-all communication. Expert popularity is skewed and input-dependent, producing load imbalance. Small decode batches provide too few tokens per expert for efficient GEMMs, while large prefill batches create large communication bursts. MoE inference can therefore trade dense compute pressure for network, memory capacity, routing, and imbalance pressure [22]–[24].

10.2 Communication model

For T tokens in a layer, hidden size d, activation precision b bytes, and top-k routing, the ideal payload for one dispatch or combine is on the order of $T \cdot k \cdot d \cdot b$, before protocol, padding, indices, and imbalance. Since MoE requires both dispatch and combine, the layer-level communication lower bound is approximately:

$$\text{Bytes_MoE,layer} \gtrsim 2 \cdot T \cdot k \cdot d \cdot b \quad (16)$$

Actual traffic can be higher because of capacity padding, duplicated routes, topology mismatch, or intermediate staging buffers. More importantly, the slowest expert or link determines layer completion. Average bandwidth is therefore less informative than the distribution of expert loads and the critical-path transfer.

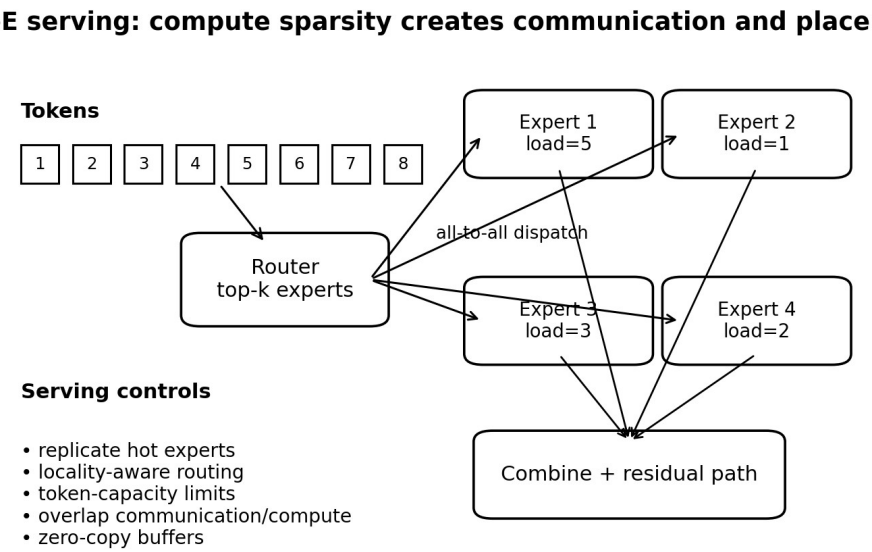


Figure 5. MoE serving converts parameter sparsity into dynamic token routing. Efficient systems must control all-to-all traffic, hot-expert imbalance, buffer movement, and small per-expert batches.

10.3 Main optimization levers

Bottleneck	Mechanism	Tradeoff
Hot experts	Replicate popular experts; adaptive placement	Consumes more memory and requires routing consistency
All-to-all volume	Lower activation precision; local routing; topology-aware assignment	May affect quality or router freedom
Staging copies	Fused/zero-copy dispatch and combine buffers	Requires tight runtime and communication-library integration
Small expert batches	Token regrouping, delayed microbatching, expert fusion	Can increase latency and complicate ordering

Load imbalance	Capacity factors, dynamic scheduling, work stealing	Dropped/rerouted tokens or extra synchronization
Network/SM interference	Overlap collectives with compute; offload movement engines	Contention must be modeled, not assumed away
Memory capacity	Expert quantization, tiering, selective residency	Misses trigger transfer or recomputation

10.4 Expert parallelism and specialized fabrics

Lina demonstrates dynamic resource scheduling to balance all-to-all transfer under skewed expert popularity [23]. SiDA-MoE explores data-aware sparsity and heterogeneous placement to reduce the memory cost of large experts [24]. More recent communication work targets redundant buffer copies and underused scale-up links; SwiftEP, for example, fuses buffers and offloads movement to dedicated mechanisms to improve MoE prefill capacity [26]. These systems point toward a hardware requirement: MoE accelerators need not only fast matrix units but also high-radix, low-overhead collectives, scatter/gather support, multicast/reduction primitives, and independent data-movement engines.

MoE also strengthens the case for phase specialization. Prefill offers many tokens that can be grouped by expert and can saturate the fabric; decode offers few tokens per sequence and may be latency-dominated. A service may choose different expert placement, replication, or routing constraints for the two phases.

11. Why Inference and Reinforcement Learning Are Converging

11.1 Rollouts are inference

In RL post-training, the policy generates trajectories that are scored and used for optimization. For language models, generation is autoregressive inference, often with multiple samples per prompt and long reasoning traces. Let B be the number of prompts per update, K the number of rollouts per prompt, and L_out the average generated length. The rollout token volume per update is approximately B·K·L_out, which can exceed the token volume used in the subsequent gradient step.

As reasoning length and sample multiplicity grow, rollout capacity becomes a primary training constraint. OpenRLHF explicitly integrates high-throughput inference engines for response generation [28], while HybridFlow treats RLHF as a distributed dataflow whose generation and training components require different sharding and orchestration [27].

$$\text{Rollout tokens per update} \approx B \cdot K \cdot E[L_{out}] \quad (17)$$

$$\text{RL throughput} \approx \text{accepted training trajectories / wall-clock time} \quad (18)$$

11.2 Serving techniques transfer—but the objective changes

Paged KV management, continuous batching, prefix caching, quantization, and speculative decoding can all accelerate rollouts. Yet the RL objective is not simply maximum token throughput. The system must preserve or record the policy distribution, maintain log probabilities and masks, associate rewards with the correct trajectory, and control policy lag between generation and optimization. A fast rollout generated by an obsolete policy may be less useful than a slower on-policy sample.

RL workloads also have structured reuse. Multiple rollouts share the same prompt, system message, retrieved context, or environment state, making prefix sharing especially valuable. Grouped sampling can share prompt KV blocks and diverge via copy-on-write. However, correlated scheduling can reduce exploration diversity if random-number streams or sampling temperatures are mishandled.

11.3 Colocation, disaggregation, and policy refresh

A colocated RL system time-shares accelerators between rollout generation and training. It avoids dedicated idle pools but pays for repeated resharding, optimizer-state movement, and phase transitions. A disaggregated system dedicates rollout workers and training workers, improving overlap but requiring frequent policy weight transfer. Hybrid designs keep a stable rollout replica and stream incremental or quantized weight updates, or periodically switch policy versions at synchronization barriers.

The policy manager should make versioning explicit. Each trajectory carries a policy identifier, sampling configuration, tokenizer version, and environment version. The learner can reject stale data, apply off-policy correction, or bound the maximum lag. Backpressure should be based on useful trajectory inventory rather than raw rollout queue length.

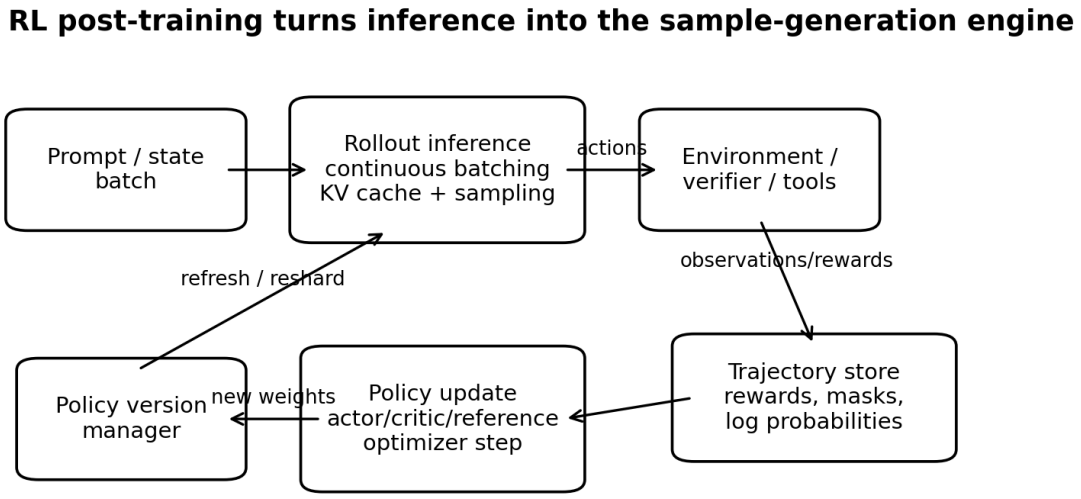


Figure 6. RL post-training contains an inference subsystem whose outputs are trajectories rather than user-visible tokens. Policy refresh, versioning, environment interaction, and metadata change the optimal scheduling objective.

11.4 Speculation and RL

Distribution-preserving speculative decoding is conceptually compatible with on-policy rollouts because accepted-and-corrected samples target the same distribution. In practice, the implementation must verify that numerical precision, rejection sampling, random-number consumption, and policy updates do not introduce silent drift. Approximate speculative methods that alter the distribution may still be useful, but they become an RL algorithm choice rather than a transparent systems optimization.

The best systems metric is useful trajectories per joule under a policy-lag and quality constraint. This metric naturally penalizes rejected speculative work, stale rollouts, invalid tool executions, and trajectories that miss a curriculum or environment deadline.

12. Specialized Inference Hardware

12.1 Why specialization is likely

General accelerators are optimized for broad programmable throughput. Large-model inference has a more specific recurring structure: low-precision matrix operations, normalization and activation functions, attention over persistent KV state, sampling, sparse expert routing, and repeated movement through a known hierarchy. Domain-specific accelerators have historically achieved major efficiency gains by matching dataflow and local memory to the workload [29], [30]. Inference creates a similar opportunity, especially when energy and latency are constrained.

The most important specialization is not simply “more matrix multiply.” Peak matrix throughput is already abundant relative to the ability to keep units fed in many decode regimes. The valuable features are those that reduce movement, increase reuse, support flexible low precision, and overlap independent resources.

12.2 Hardware design priorities

Priority	Hardware implication	System-level effect
Locality	Large software-managed SRAM; register-rich tiles; cache residency controls	Fewer HBM trips for weights, scales, metadata, and attention tiles
Low precision	Native INT8/INT4/FP8/FP4 paths; mixed	Lower bytes and higher arithmetic density

	accumulation; fast conversion	without conversion bottlenecks
KV first-class support	Paged gather, append, quantize/dequantize, copy-on-write, prefix sharing	Reduces attention and allocator overhead; enables long-context capacity
Data movement engines	Asynchronous DMA, gather/scatter, multicast, reduction, compression	Overlap communication with compute and reduce SM consumption
MoE collectives	High-radix all-to-all, topology-aware routing, zero-copy buffers	Lowers expert dispatch/combine latency and imbalance penalty
Phase specialization	Different compute/bandwidth/capacity points for prefill and decode	Better energy and cost matching; independent scaling
Fast scale-up fabric	High bandwidth, low latency, strong collectives within a node/domain	Enables model parallelism and KV transfer without rack-network penalties
Observability	Per-engine bandwidth, queue, stall, and energy counters	Allows scheduler and compiler to optimize real bottlenecks

12.3 Memory-centric architectures

A memory-centric inference chip should keep frequently reused data near computation and make state movement explicit. Weight-stationary dataflow is attractive when batch reuse is high. KV-stationary or attention-stationary tiles are attractive for decode and long context. Near-memory transforms can quantize, transpose, pack, or route data before it consumes expensive core bandwidth. Large on-package memory and 3D integration increase capacity and bandwidth but do not eliminate the need for reuse: every off-tile access still consumes energy and can create contention.

The software interface matters. If KV paging, prefix sharing, expert routing, and precision metadata are hidden behind generic tensor loads, the compiler cannot reliably exploit the hardware. Future instruction sets or runtime APIs should expose block-level KV operations, sparse token routes, asynchronous collectives, and phase intent.

12.4 Extremely low precision

Low precision reduces both arithmetic and movement, but inference hardware must support the complete numeric path. This includes per-group scale loads, zero points, outlier lanes, mixed-precision accumulation, normalization, softmax, logits, and conversion. A nominal FP4 or INT4 multiply engine is not sufficient if dequantization runs on a scarce general-purpose pipeline or if intermediate tensors are repeatedly expanded to FP16 in memory.

The most promising design is fused precision flow: compressed weights and KV state are fetched, transformed near the consumer, accumulated at a safe precision, and never materialized in an unnecessarily wide intermediate format. Precision should be programmable by tensor, layer, channel group, age of KV block, or phase.

12.5 Separate silicon or separate operating points?

Prefill and decode can justify different chips, but they can also share a heterogeneous package. A compute-dense prefill engine, a bandwidth-rich decode engine, a large KV memory tier, and a communication engine may be integrated as chiplets. The decision depends on utilization and transfer costs. Separate devices are attractive when workloads are large enough to keep each pool busy and the interconnect can carry KV state efficiently. Shared silicon is attractive for small deployments, bursty phase ratios, and short prompts where transfer overhead is difficult to amortize.

A practical near-term path is specialization through configurable operating points: partitionable compute arrays, independently clocked memory and movement engines, flexible precision, and runtime-controlled SRAM allocation. The long-term path may be genuinely phase-specific accelerators connected by a KV-aware fabric.

13. A Reference Inference Architecture

Figure 4 summarized a reference architecture whose organizing abstraction is state placement. The architecture is not a prescription that every deployment must fully disaggregate. It is a decomposition that makes costs explicit and allows components to be colocated when locality is more valuable than separation.

13.1 Components

1. Ingress and workload classifier. Extract model, adapter, prompt length, expected output class, latency tier, sampling mode, and RL policy version.
2. Global admission controller. Enforce memory-safe concurrency and SLO-aware backpressure before work consumes scarce KV blocks.
3. Prefix and KV directory. Track token-prefix identity, block ownership, precision, tier, reference count, and transfer state.
4. Prefill pool. Optimize large prompt batches, chunking, prompt attention, and first-token latency.
5. Decode pool. Optimize continuous batching, KV reads, weight reuse, speculation verification, and streaming cadence.
6. Transfer and cache fabric. Move KV blocks among GPU memory, CPU memory, storage, and remote workers with topology awareness.
7. Speculation service. Generate drafts, select proposal depth, and record acceptance by workload class.
8. MoE expert fabric. Place and replicate experts, route tokens, and schedule all-to-all communication.
9. RL rollout coordinator. Manage policy versions, environment sessions, trajectory metadata, and synchronization with learners.
10. Observability and optimizer. Maintain cost models from real traces and choose operating points on the latency-throughput-energy frontier.

13.2 Control loop

The scheduler should operate as a feedback controller. It observes queue lengths, prompt and output token rates, KV occupancy, memory bandwidth, compute utilization, interconnect congestion, expert imbalance, speculation acceptance, and SLO violations. It then adjusts concurrency caps, prefill chunk size, decode token budget, pool ratios, precision modes, and routing. Static tuning is inadequate because workload distributions and model versions change.

The controller must also avoid oscillation. Rapidly moving capacity between prefill and decode can cause alternating queue buildup. Hysteresis, minimum dwell times, predictive scaling, and multi-timescale control are useful: microsecond decisions form batches; millisecond decisions route requests; second-scale decisions change pool allocations; minute-scale decisions autoscale nodes and migrate hot prefixes.

13.3 Security and isolation

KV sharing and prefix caching introduce isolation requirements. Shared physical blocks must be keyed by model, tokenizer, exact token sequence, adapter, precision, and security domain. Reference counts and copy-on-write must prevent one tenant from observing or corrupting another’s state. Cache timing and occupancy can create side channels, so sensitive deployments may require partitioned caches, randomized eviction, or disabled cross-tenant reuse. RL trajectories can contain private prompts, tool outputs, and reward signals and should follow the same data-governance controls as training data.

14. Measurement and Optimization Methodology

14.1 Start with workload distributions

Optimization should begin with traces, not kernels. Measure prompt length, output length, arrival burstiness, tenant mix, prefix repetition, sampling configuration, model/adapter popularity, tool-call frequency, and latency tier. Report correlations: long prompts may also produce long outputs; certain prefixes may dominate peak hours; RL prompts may request multiple samples. The joint distribution determines queueing and state residency.

14.2 Build a phase-specific cost model

Measure	Prefill	Decode	MoE / distributed	RL rollout
Work unit	Prompt token or chunk	Generated token	Routed token / collective	Accepted trajectory
Latency	TTFT service component	TPOT / iteration	Dispatch-compute-combine critical path	Generation + environment step
State	Prompt activations + KV creation	Growing KV cache	Expert weights + route buffers	KV + trajectory metadata
Bandwidth	Weights, activations,	Weights + KV reads	All-to-all + reductions	Weights/KV + policy

	attention			refresh
Quality signal	First-token correctness	Sampling fidelity	Router/expert quality	Reward and policy distribution

14.3 Attribute bytes and stalls

For each phase, attribute time and bytes to weights, KV state, activations, collectives, host transfers, and control overhead. Hardware counters should distinguish compute-active time, memory stalls, interconnect stalls, launch gaps, and synchronization. Kernel-only profiles should be reconciled with end-to-end traces; otherwise scheduler gaps and queueing disappear from the analysis.

A useful experiment holds one dimension constant while sweeping another: batch size at fixed sequence length, context length at fixed concurrency, quantization at fixed quality, prefill chunk size at fixed TPOT SLO, or speculation depth at fixed acceptance class. Plot both throughput and tail latency. When the curve changes slope, the bottleneck has likely moved.

14.4 Optimization order

11. Correctness and quality contract: verify outputs, sampling distribution, numerical behavior, and isolation.
12. Eliminate gross waste: padding, allocator fragmentation, duplicate prefixes, unnecessary copies, idle batch slots, and oversized reservations.
13. Improve locality: paging, prefix sharing, fusion, tiling, weight reuse, KV layout, and expert placement.
14. Reduce representation width: apply phase-appropriate quantization to weights, activations, KV state, and communication.
15. Overlap resources: compute with memory, collectives with expert computation, and rollout generation with training.
16. Disaggregate only when specialization and queueing gains exceed transfer and replication costs.
17. Specialize hardware after the stabilized workload reveals which movement and control operations remain dominant.

14.5 Reporting checklist

- Model architecture, active parameter count, attention type, KV heads, precision, context limit, and parallelism.
- Hardware model, memory capacity/bandwidth, interconnect topology, software versions, and power measurement method.
- Prompt/output distributions, arrival process, concurrency, prefix reuse, and sampling settings.
- TTFT, TPOT, end-to-end latency, p50/p95/p99, request throughput, token throughput, and SLO goodput.
- KV occupancy and fragmentation, cache hit rate, bytes moved at major boundaries, and batch-size distribution.
- Speculative acceptance length, rejected work, draft cost, and target verification cost.
- For MoE: expert load distribution, all-to-all bytes, critical-path link utilization, and expert batch sizes.
- For RL: policy lag, rollout length, trajectories accepted per update, environment time, and weight-refresh overhead.
- Energy or cost normalized by useful tokens or useful trajectories.

15. Research Agenda

15.1 A universal state abstraction

Current systems expose weights, KV caches, prefixes, speculative branches, and RL trajectories through separate mechanisms. A unified state abstraction could describe immutable shared blocks, mutable sequence tails, precision, ownership, version, and placement. Such an abstraction would allow the runtime to move or share state across prefill, decode, agents, and training without ad hoc copies.

15.2 Learned but safe scheduling

Schedulers could learn output-length distributions, speculation acceptance, prefix reuse, expert popularity, and queueing response. The challenge is safety: memory oversubscription and SLO collapse are unacceptable. Promising directions combine learned prediction with hard feasibility constraints, robust uncertainty margins, and verifiable admission control.

15.3 Precision as a runtime resource

Precision is usually fixed at model load time. Future systems could select precision by phase, layer, request tier, KV age, network congestion, or RL exploration stage. This requires calibrated error budgets and hardware that switches formats

without expensive conversions. The scheduler would treat precision as another resource knob alongside batch size and placement.

15.4 Cross-request and cross-trajectory reuse

Prefix caching is the first form of reuse, but inference programs contain richer common subgraphs: shared tool schemas, retrieved documents, verification prefixes, agent plans, and environment states. Detecting and safely reusing these structures could reduce prefill and rollout cost. The research problem combines semantic equivalence, exact token identity, cache consistency, and privacy.

15.5 MoE locality-aware training

Serving systems currently compensate for routing patterns chosen during training. A stronger co-design would train routers with topology, expert replication cost, and latency in the objective. The challenge is to preserve model quality and load balance without collapsing expert diversity. Hardware-aware routing could reduce all-to-all traffic at its source rather than optimizing transport after the fact.

15.6 Inference-native RL algorithms

RL algorithms can be redesigned around rollout-system constraints. Partial rollouts, early termination, adaptive sample multiplicity, uncertainty-driven generation, and trajectory reuse can reduce inference cost. The algorithm should value information gained per generated token, while the system should expose real-time generation cost and policy lag. This is a joint learning-systems optimization problem.

15.7 Memory-semantic hardware

The most consequential specialized hardware may understand inference state rather than generic tensors. A memory-semantic accelerator could append and gather KV pages, maintain reference counts, perform copy-on-write, quantize old blocks, execute prefix comparisons, and move expert tokens without routing every operation through general cores. Such support would reduce both bytes and control overhead, but requires stable software abstractions and strong isolation.

16. Conclusion

Large-model inference is the point at which algorithmic capability meets physical constraints. The service must repeatedly deliver model weights and growing attention state to arithmetic units, schedule variable-length requests, preserve token cadence, and coordinate distributed computation. Every major optimization discussed in this paper changes the data path. Paged attention changes KV allocation and sharing. Continuous batching changes weight reuse. Speculative decoding trades cheap proposal work for fewer expensive target passes. Quantization compresses movement and enables lower-energy arithmetic. Prefill/decode disaggregation trades locality for isolation and specialization. MoE trades dense compute for dynamic communication. RL post-training turns rollout inference into a central training workload.

The broad claim that data movement can cost orders of magnitude more energy than arithmetic is directionally correct but should not be used as a substitute for measurement. The bottleneck depends on batch size, context, precision, topology, kernels, and overlap. The engineering objective is to measure the current limiting resource and deliberately reduce or hide its cost until another resource becomes binding.

The next generation of inference systems will therefore be co-designed across model architecture, numerical format, memory hierarchy, scheduler, interconnect, and silicon. Their success will be measured by SLO-compliant goodput, useful tokens per joule, and useful trajectories per accelerator-hour. Peak FLOP/s will remain relevant, but it will no longer be the defining metric. The defining capability will be how intelligently the platform places, reuses, compresses, and moves state.

References

[1] A. Vaswani et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems, 2017.

[2] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” Communications of the ACM, vol. 52, no. 4, 2009.

[3] M. Horowitz, “Computing’s Energy Problem (and What We Can Do About It),” IEEE International Solid-State Circuits Conference, 2014.

[4] S. Han et al., “EIE: Efficient Inference Engine on Compressed Deep Neural Network,” International Symposium on Computer Architecture, 2016.

[5] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” Advances in Neural Information Processing Systems, 2022.

[6] G.-I. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” USENIX Symposium on Operating Systems Design and Implementation, 2022.

[7] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” ACM Symposium on Operating Systems Principles, 2023.

[8] J. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” Conference on Empirical Methods in Natural Language Processing, 2023.

[9] Z. Liu et al., “KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache,” International Conference on Machine Learning, 2024.

[10] L. Zheng et al., “SGLang: Efficient Execution of Structured Language Model Programs,” Advances in Neural Information Processing Systems, 2024.

[11] A. Agrawal et al., “Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve,” USENIX Symposium on Operating Systems Design and Implementation, 2024.

[12] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” International Conference on Machine Learning, 2023.

[13] C. Chen et al., “Accelerating Large Language Model Decoding with Speculative Sampling,” arXiv:2302.01318, 2023.

[14] G. Xiao et al., “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models,” International Conference on Machine Learning, 2023.

[15] E. Frantar et al., “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” International Conference on Learning Representations, 2023.

[16] J. Lin et al., “AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration,” Proceedings of Machine Learning and Systems, 2024.

[17] Y. Lin et al., “QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving,” Proceedings of Machine Learning and Systems, 2025.

[18] Y. Zhong et al., “DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving,” USENIX Symposium on Operating Systems Design and Implementation, 2024.

[19] P. Patel et al., “Splitwise: Efficient Generative LLM Inference Using Phase Splitting,” International Symposium on Computer Architecture, 2024.

[20] R. Qin et al., “Mooncake: Trading More Storage for Less Computation—A KVCache-Centric Architecture for Serving LLM Chatbot,” USENIX Conference on File and Storage Technologies, 2025.

[21] K. Zhu et al., “NanoFlow: Towards Optimal Large Language Model Serving Throughput,” USENIX Symposium on Operating Systems Design and Implementation, 2025.

[22] S. Rajbhandari et al., “DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale,” International Conference on Machine Learning, 2022.

[23] J. Li et al., “Accelerating Distributed MoE Training and Inference with Lina,” USENIX Annual Technical Conference, 2023.

[24] Z. Du et al., “SiDA: Sparsity-Inspired Data-Aware Serving for Efficient and Scalable Large Mixture-of-Experts Models,” Proceedings of Machine Learning and Systems, 2024.

[25] A. Q. Jiang et al., “Mixtral of Experts,” arXiv:2401.04088, 2024.

[26] X. Li et al., “SwiftEP: Accelerating MoE Inference with Buffer Fusion and TMA Offloading,” USENIX Symposium on Networked Systems Design and Implementation, 2026.

[27] G. Sheng et al., “HybridFlow: A Flexible and Efficient RLHF Framework,” European Conference on Computer Systems, 2025.

[28] J. Hu et al., “OpenRLHF: A Ray-Based Easy-to-Use, Scalable and High-Performance RLHF Framework,” Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2025.

[29] N. P. Jouppi et al., “In-Datacenter Performance Analysis of a Tensor Processing Unit,” International Symposium on Computer Architecture, 2017.

[30] Y.-H. Chen, T. Krishna, J. Emer, and V. Sze, “Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks,” IEEE Journal of Solid-State Circuits, 2017.

[31] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” International Conference on Learning Representations, 2024.